

DSA STUDY BLUEPRINT SERIES

18. Sort Array 0s 1s 2s DNF Algorithm

Sorting 0s, 1s, and 2s in Linear Time

Section 03 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Dutch National Flag:** Uses three pointers: `low`, `mid`, and `high`.
- Partition array boundaries such that all 0s are $< \text{low}$, 1s between `low` & `mid`, 2s $> \text{high}$.

Memory & Execution Blueprint



Red (0s) $\leftarrow$ [Low Ptr] | White (1s) $\leftarrow$ [Mid Ptr] | Blue (2s) $\leftarrow$ [High Ptr]

C++ Implementation Reference

Standard code layout and syntax

```
void sort012(vector& nums) {  
    int low = 0, mid = 0, high = nums.size() - 1;  
    while (mid <= high) {  
        if (nums[mid] == 0) swap(nums[low++], nums[mid++]);  
        else if (nums[mid] == 1) mid++;  
        else swap(nums[mid], nums[high--]);  
    }  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N)$ (Single Pass)
Space Complexity	$O(1)$

Key Practices & Gotchas

- **Important:** Avoid counting frequency and writing; DNF performs elements swapping in a single pass.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run the Dutch National Flag 3-pointer partition step for `[2, 0, 1]` :

Iteration	Pointers [low, mid, high]	Value at mid	Swap & Pointer Adjustments	Array State
1	[0, 0, 2]	nums[0] = 2	Swap nums[mid] & nums[high]; high--	[1, 0, 2]
2	[0, 0, 1]	nums[0] = 1	nums[mid] is 1; mid++	[1, 0, 2]

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Three-way Partitions:** Partitions arrays into three zones (less, equal, greater) in a single pass.
- **Next Permutation:** Find peak breakpoint, swap elements with next-largest suffix value, and reverse suffix inline.
- **Related LeetCode Problems:** #75 Sort Colors, #31 Next Permutation.